Notes on Complexity Theory

Last updated: 2026-10-18

These notes are primarily based on the book *Computational Complexity: A Modern Approach* by Sanjeev Arora and Boaz Barak.

Contents

1 Basic Complexity Classes 1
1.1 Turing Machines 1
1.1.1 Modelling Computation 1
1.1.2 Universality 2
1.2 Uncomputability 3
1.3 The complexity class P 4

1 Basic Complexity Classes

1.1 Turing Machines

1.1.1 Modelling Computation

The Turing Machine is a useful model of computation that allows us to formally define complexity classes. In particular, any other physically realisable model of computation can be simulated by a Turing Machine with little loss of efficiency (ignoring the possibility of quantum computation, which we will address later in these notes).

We now formally define a Turing Machine.

Definition 1.1 (Turing Machine) A Turing Machine is described by a tuple (Γ, Q, δ) containing:

- A finite set Γ of symbols, called the *tape alphabet*. We assume for simplicity that Γ contains a special blank symbol $\square$, a special start symbol $\triangleright$, and the numbers 0 and 1.
- A finite set Q of states in which the TM's finite control can be in. We assume that Q contains a special start state q_{start} and a special halting state q_{halt}.
- A transition function $\delta : Q \times \Gamma^k \to Q \times \Gamma^{k-1} \times \{L, S, R\}^k$, where $k \geq 2$ (see Remark 1.2).

The Turing Machine operates as follows. It has k tapes, each of which is infinite towards the right. The first tape is the input tape, which contains the input string x on its leftmost cells and is blank elsewhere. The other tapes are work tapes, which are all initially blank. The TM's finite control is initially in the start state q_{start} and the tape heads are all positioned at the leftmost cell of their respective tapes. At each step, the TM reads the symbols under the tape heads, applies the transition function δ to determine the next state, what symbols to write on each tape, and how to move each tape head (left, stay, right). The TM continues to operate until it reaches the halting state q_{halt}, at which point

it stops. If the TM tries to go left from the leftmost cell of a tape, it remains in the same position.

Remark 1.2 This is the definition of a multi-tape Turing Machine, where the first tape is the input tape and the other tapes are work tapes. It is also possible to define a single-tape Turing Machine but it is known that multi-tape Turing Machines are equivalent to single-tape Turing Machines up to quadratic slowdown.

Definition 1.3 (Computing a function and running time) Let $f : \{0,1\}^* \to \{0,1\}^*$ and let $T : \mathbb{N} \to \mathbb{N}$ be some functions, and let M be a TM. We say that M *computes* f is for every input $x \in \{0,1\}^*$, M halts with $f(x)$ written on its output tape[1]. If the number of steps that M takes to halt on input x is at most $T(|x|)$ for every input x, we say that M *runs in time* $T(n)$.

1.1.2 Universality

Clearly we can represent TMs by binary strings. We can do this by encoding each element of the TM (Γ, Q, δ) as a binary string, and then using a binary encoding of a tuple to encode the entire TM. Furthermore, for simplicity, we would like the property that every binary string $x \in \{0,1\}^*$ represents a valid TM. We can easily achieve this by assigning all invalid binary strings to a trivial TM that immediately halts without doing anything.

With this, we can define a *Universal Turing Machine* U that takes as input (M, x), where M is understood to be the binary encoding of a TM[2] and x is an input string. The universal TM then simulates M on input x, and halts with the same output as M would have on input x.

The existence of an *efficient* universal Turing Machine is a fundamental result. In particular, we have the following theorem.

Theorem 1.4 (Efficient Universal Turing Machine) There exists a universal Turing Machine U such that for every TM M and input x, if M runs in time $T(|x|)$, then U runs in time $cT(|x|)^2$ for some constant c that depends on M but not on x.

Proof sketch. The idea is to have U simulate M step by step. We first start by converting M into a single-tape TM $\widetilde{M}$ (see Remark 1.2). This can incur a quadratic slowdown. We can also convert M into a machine with the alphabet $\{\square, \triangleright, 0, 1\}$. From now on, we can assume that M is a single-tape TM with the alphabet $\{\square, \triangleright, 0, 1\}$.

We then construct the universal TM U as follows. U first decodes its input (M, x) onto the following tapes:

[1] We can adopt the convention that the output tape is the first work tape, and that the input tape is read-only.

[2] We are using an abuse of notation here by using M to mean both the TM and the binary string representing the TM

- The first tape contains the input x. This will represent the work tape of M (remember, we are assuming that M is a single-tape TM).
- The second tape contains the transition function δ of M.
- The third tape contains the current state of M. This is initially set to the start state q_{start}.

To simulate one computational step of M, U first reads the current state of M from the third tape, and then reads the symbol under the first tape head (which is the work tape of M). It then scans through the second tape to find the corresponding transition in δ. This takes time proportional to the size of δ, which is a constant that depends on M but not on x. Once it finds the transition, it updates the first and third tapes accordingly, and moves the first tape head according to the transition. This simulates one step of M. □

Remark 1.5 Theorem 1.4 can be strengthened to show that there exists a universal TM that runs in time $cT(|x|)\log T(|x|)$.

1.2 Uncomputability

Theorem 1.6 There exists a function $f : \{0,1\}^* \to \{0,1\}^*$ that is not computable by any TM.

Proof. We can easily construct an explicit example of such a function.

Let f be defined as follows. For every binary string $x \in \{0,1\}^*$, let $f(x) = 1$ if the TM represented by x halts on input x and returns 0. Otherwise, let $f(x) = 0$ (i.e. if the TM represented by x does not halt on input x, or if it halts and returns something else other than 0).

Now suppose for contradiction that there exists a TM M that computes f. Let x_M be the binary string representing M. We have two cases:

- If M halts on input x_M and returns 0, then by the definition of f, we have $f(x_M) = 1$. But since M computes f, we also have $f(x_M) = M(x_M) = 0$, which is a contradiction.
- If M does not halt on input x_M, or if it halts and returns something other than 0, then by definition of f, we have $f(x_M) = 0$. But since M computes f, we also have $f(x_M) = M(x_M) = 1$, which is a contradiction.

So in either case, we reach a contradiction. Therefore, no TM can compute f. □

Example 1.7 A more natural example of an uncomputable function is the *halting problem*, which is the function $\mathsf{HALT} : \{0,1\}^* \to \{0,1\}$ which returns 1 if the TM represented by the input string halts on that input, and returns 0 otherwise.

If we had a TM M_{Halt} that computes HALT, we could use it to compute the function f defined in the proof of Theorem 1.6, which is a contradiction. The idea is that we can use M_{Halt} to first determine whether some TM M_x halts on input x. If it does not halt, we can

immediately return 0. If it does halt, we can simulate M_x on input x using the universal TM to determine what it returns.

Therefore, the halting problem is uncomputable.

1.3 The complexity class P

A common framework for studying the (time) complexity of problems is in terms of *languages* and *decision problems*. A language L is a subset of $\{0,1\}^*$ and a decision problem is a problem that can be phrased as a question of whether a given string $x \in \{0,1\}^*$ belongs to some language L. Informally, we can think of decision problems as yes/no questions about strings.

We say that a TM M *decides* a language L if M computes $f_L : \{0,1\}^* \to \{0,1\}$ where $f(x) = 1 \Leftrightarrow x \in L$. f_L is also called the *characteristic function* of L.

Definition 1.8 (The class DTIME(t))